Digital Twin and Performance Monitoring System for a Ball and Spring Model

George Francis

June 2026

Abstract

I. INTRODUCTION

Digital twins are an infant concept in engineering which have already displayed optimistic uses in a variety of fields ranging from smart cities to medicine (1). A digital twin is a 'twin' of am existing object which monitors and analyses information of each part to evaluate predictive lifecycle management such as servicing and replacing parts. This technology is imperative in engineering for determining and assisting in machine maintenance. Recent advancements in digital twin technology has been observed in medicine (2) by which digital twins of humans have attempted to be created to monitor organ function and predict failure or maintenance. An extremely recent study using digital twin technology (3) has developed a python module which can help monitor tumour growth called TumourTwin. This technology is still in its infant phase and yet is already showing promising signs of being a vital part of society. This paper is simply an endeavour of curiosity alongside a logbook for what I learn and apply. I aim to explore and apply digital twin technology using simple well known physics models in python, such as a ball and spring to gain an understanding of how this technology works.

II. METHOD

To begin development of a digital twin it is first important to develop the system that we intend to make a twin of. Python was used to create a simple ball-spring model with a damping constant ε alongside Newtons second law for a spring system

$$m\ddot{x} + c\dot{x} + kx = 0 \tag{1}$$

Where 'm' is the mass of the ball, 'c' is the damping constant and 'k' is the spring constant. 'x' in this instance refers to the displacement of the ball. Using this simple equation we can model an oscillating ball and spring model which generates the following result

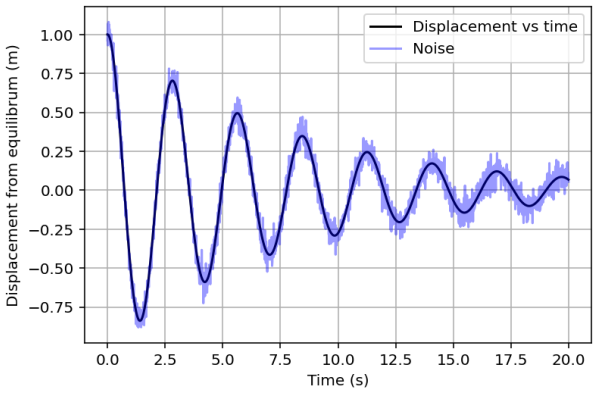

Figure I: Oscillating ball and spring model with a ball of mass 1kg over 20 seconds and a damping constant of $0.3Ns/m$. Seen behind the theoretically perfect model is a noisy model which is seen to deviate from the perfect model slightly

Notice in figure (I) there is noisy data which is included to display results more accurate to a real physical system. Measurement noise can occur from a variety of sources such as electronic interference or general errors in the equipment. So whilst the noise isn't completely analogous to real life, it still indicates similarities that a real system would display. To create this model (See Appendix A.) initial conditions were set and a function was created to solve two first order differential equations as we cant solve a second order differential in a loop using python.

$$\dot{x} = v \tag{2}$$

$$v = \frac{1}{m}\left(-kx - cv\right) \tag{3}$$

The Euler time stepping method is then used to evolve this function in which can be visualised mathematically as

$$x_{i+1} = x_i + \dot{x} \cdot \Delta t$$

$$v_{i+1} = v_i + \dot{v} \cdot \Delta t$$

This gives us the real reading of the spring. Then Gaussian noise is used to distort the reading to give our 'sensor reading' where

$$z(t) = x(t) + \epsilon$$

$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

From this another python file is made where a Kalman filter (4) is coded to estimate future movements of the system based on the noisy data that it has been given. The Kalman filter tracks two values rather than one so matrices are used in python to estimate values. It compares the sensor readings to the model and calculates a value '$\mathcal{K}$' known as the Kalman gain where

$0 \leq \mathcal{K} \leq 1$, a value of 1 corresponds to an accurate sensor reading and a value of 0 corresponds to a bad sensor reading. It is important to note that we use state representation

$$\mathbf{x} = [x, v]^T$$

Using this notation a transiton matrix is created

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t \\ \left(-\frac{k}{m}\right)\Delta t & 1 - \left(\frac{c}{m}\right)\Delta t \end{bmatrix}$$

Which encodes the Euler update method into the matrix, so if we multiply $\mathbf{F}$ by the predicted state we get the predicted next state. Then a Kalman update function is created which essentially returns a better estimate. We then use this and time step it to overlay the Kalman filter estimate over the top of the original graph.

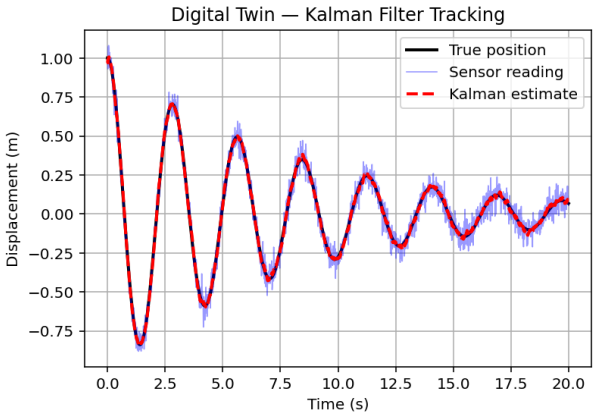

Figure II: Mass-spring system indicating the true position (black), the noisy data to account for measurment errors (blue) and Kalman filter estimate (red)

Figure (II) displays a very accurate Kalman filter reading on the oscillation. Then we determine what would happen if a fault injection was imposed on the model to see how the Kalman filter would react. To do this, the damping constant 'c' was changed from 0.3 to 3 after 5 seconds. This creates the following plot:

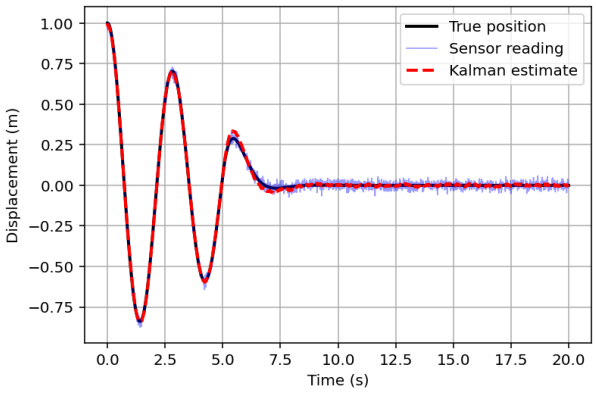

Figure III: Mass-spring system with fault injection imposed at T=5. Graph displays Kalman filter estimate (Red), noisy data (blue)

Figure (III) indicates that the fault injection severely increases the damping of the model. We can visualise this divergence by creating a subplot of residuals to see how much the Kalman filter differs each estimate.

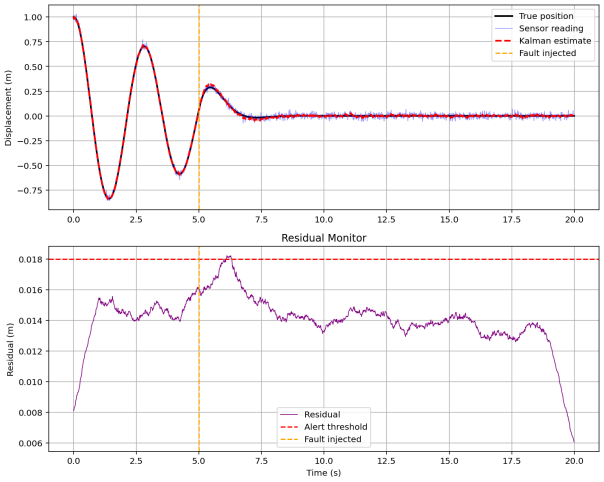

Figure IV: Residual plot of kalman filter against spring-ball system with fault injection imposed at T=5

III. RESULTS

Analysis of the Kalman filter in figure (II) Indicate a highly accurate reading due to minimal divergence from the actual movement of the system. When a fault injection is imposed, it can be seen that there is a slight divergence, the Kalman filter quickly adjusts to the change and accounts for the fault injection as seen in figure (III) again, indicating a highly accurate filter.

IV. CONCLUSION

Through this experiment I have documented my journey regarding how I learned how to create and code a digital twin. The use of this technology is extremely exciting to me and I believe it will imperative in future advancements. I began this project by researching what a digital twin is, I then applied my physics knowledge to create a mass-spring system and created an oscillation simulation in python. Then I created the digital twin in a separate python file which displayed accurate measurements and showed promising results for fault injections.

References

[1] Botín-Sanabria, D.M. et al. (2022) 'Digital Twin Technology Challenges and Applications: A comprehensive review', Remote Sensing, 14(6), p. 1335. doi:10.3390/rs14061335.

[2] Sun, T., He, X. and Li, Z. (2023) 'Digital Twin in healthcare: Recent updates and challenges', DIGITAL HEALTH, 9. doi:10.1177/20552076221149651.

[3] Kapteyn, M.G. et al. (2026) 'TumorTwin: A python framework for patient-specific digital twins in oncology', BMC Medical Informatics and Decision Making [Preprint]. doi:10.1186/s12911-026-03520-2.

[4] Naya, M.Á. et al. (2023) 'Kalman filters based on multibody models: Linking simulation and Real World. A comprehensive review', Multibody System Dynamics, 58(3–4), pp. 479–521. doi:10.1007/s11044-023-09893-w.